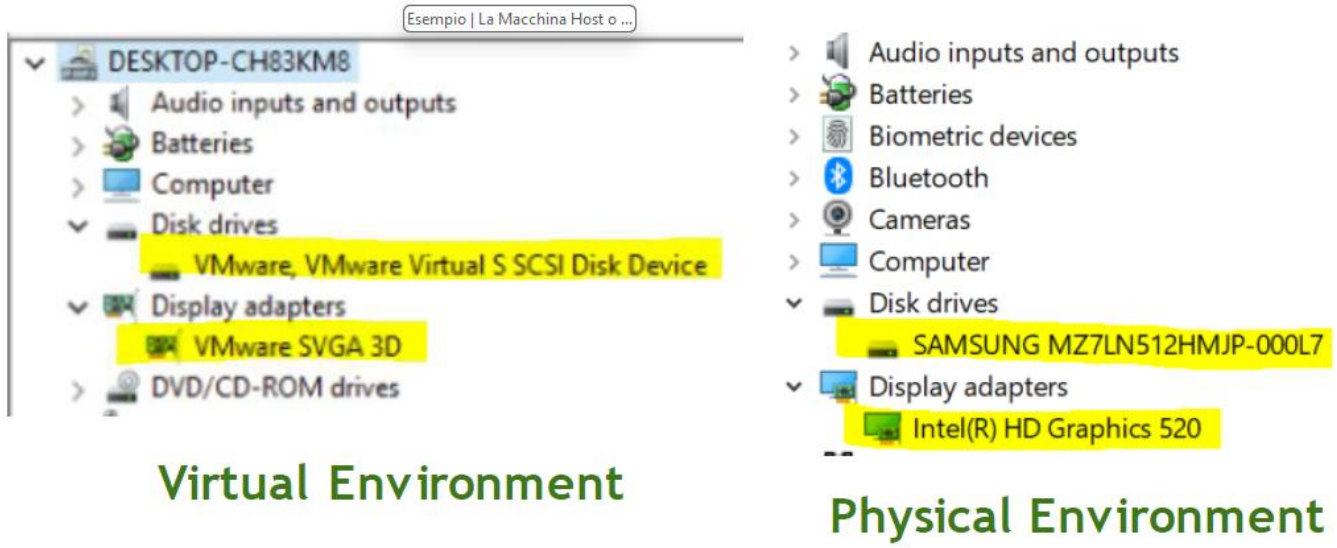
| DOCENTE | Paolo BELLONI [paolo.belloni@its-ictpiemonte.it] |
|---|---|
| Unità Formativa | **Sistemi di Virtualizzazione** |
| Argomento | La Virtualizzazione a livello HW |

Intervento realizzato da ITS TECNOLOGIE DELL'INFORMAZIONE E DELLA COMUNICAZIONE

# Esempio | Macchina fisica vs virtuale

## Analisi con il Windows Device Manager




- Viene confrontato l'output di Windows Device Manager di una virtual machine e di un laptop fisico.
- Si può vedere dove reale e virtuale iniziano a divergere.
- L'esame dell'adattatore di rete e dell'adattatore del disco rivelano i dispositivi standard ma generici.
- L'adattatore per il display non è lo stesso di un monitor reale. Viene creato come driver di dispositivo standard da utilizzare su qualsiasi monitor.

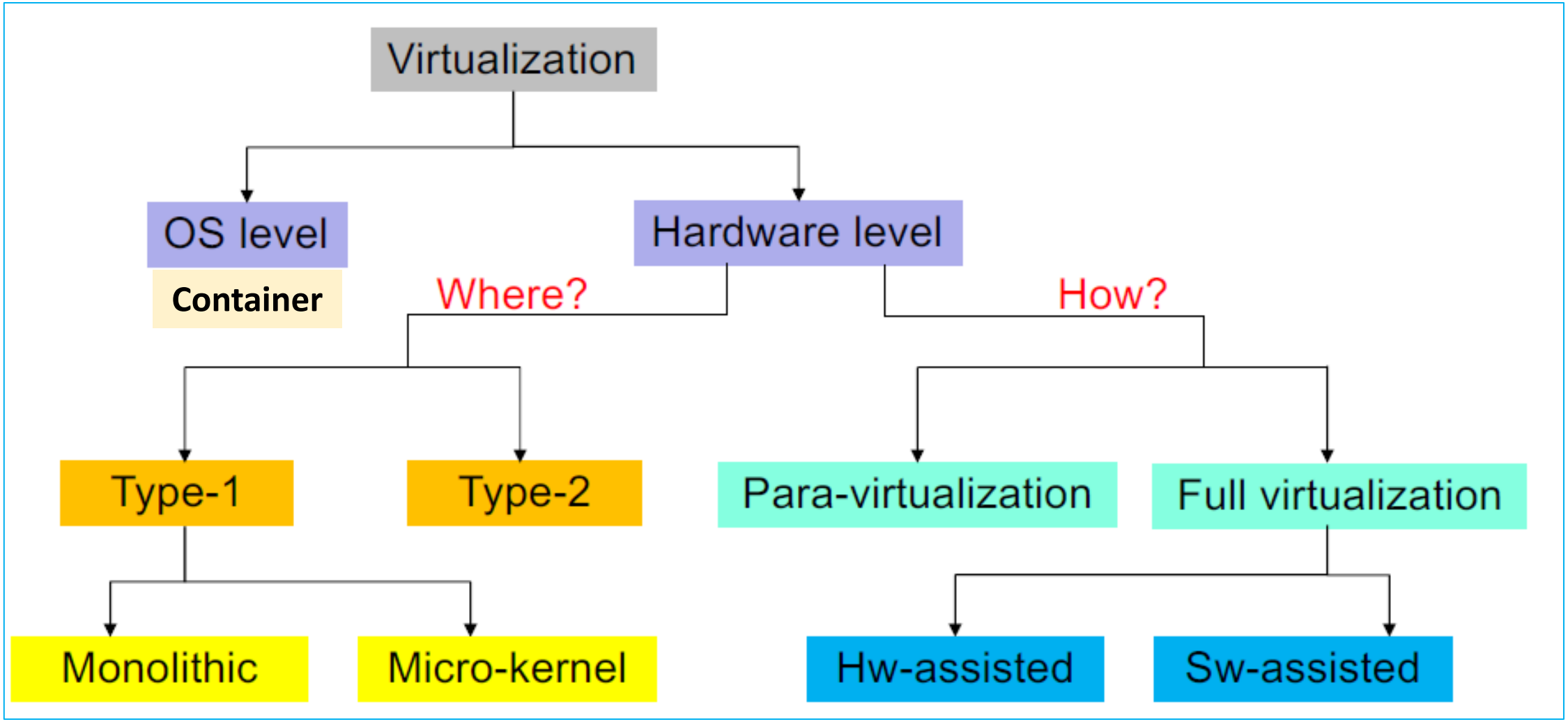
# Classificazione della Virtualizzazione a livello HW

Esistono diversi tipi di virtualizzazione a livello HW (o system level virtualization), distinti in base a:

- Dove si istanza l'hypervisor:
  - System hypervisor, o di sistema, chiamato anche type-1, native or bare-metal hypervisor;
  - Hosted hypervisor, o ospitato, chiamato anche type-2 hypervisor.

- Come si virtualizzano le istruzioni sensibili e non virtualizzabili.
  - Full virtualization:
    - Software-assisted;
    - Hardware-assisted.
  - Para-virtualization.

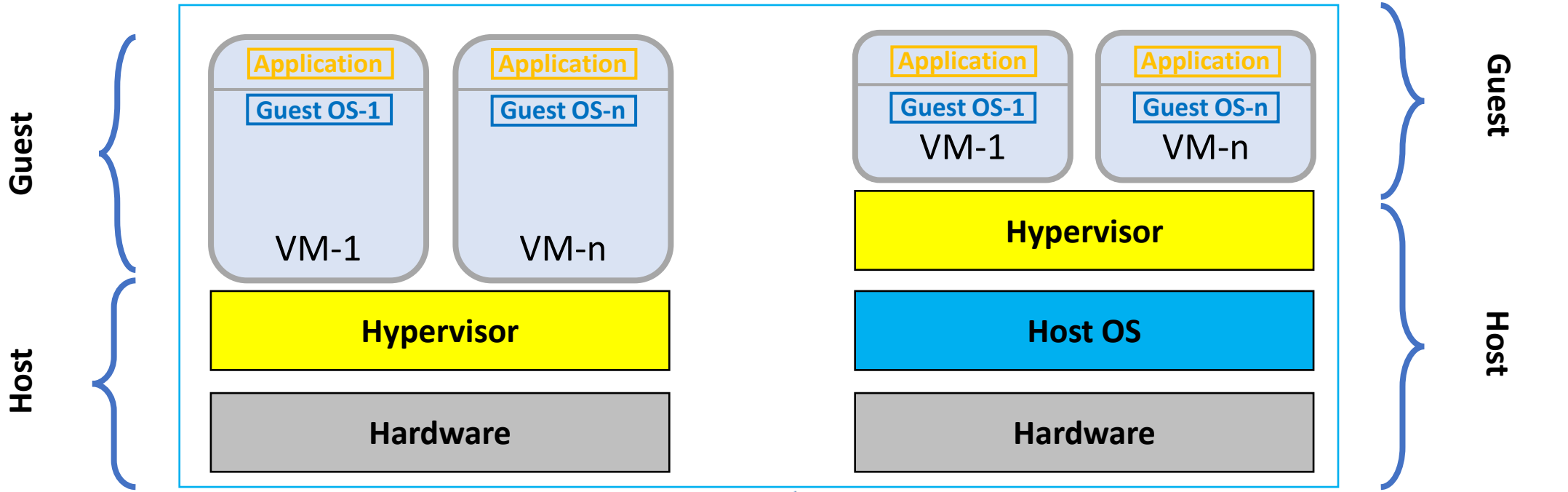
# Classificazione della Virtualizzazione a livello HW



# Hypervisor di Sistema vs Ospitato 1/2

In quale livello dell'architettura di sistema si colloca l'hypervisor?

- Direttamente sull'hardware: **hypervisor di sistema**
- Sopra il SO host: **hypervisor ospitato**




Hypervisor di sistema

Hypervisor ospitato

# Hypervisor di Sistema vs Ospitato 2/2

- **Hypervisor di sistema (type-1)**: eseguito direttamente sull'hw, offre funzionalità di virtualizzazione integrate in un SO semplificato (è, di fatto, un SO).
  - L'hypervisor può avere un'architettura a micro-kernel (solo funzioni di base, no device driver) o monolitica.
  - Richiede HW certificato.
  - Esempi: Xen, KVM (non univocamente riconosciuto), VMware ESX, Hyper-V.
- **Hypervisor ospitato (type-2)**: eseguito sul SO host, accede alle risorse hw tramite le chiamate di sistema del SO host.
  - Interagisce con il SO host tramite l'ABI ed emula l'ISA di hw virtuale per i SO guest.
  - Vantaggio: può usare il SO host per gestire le periferiche ed utilizzare servizi di basso livello (es. scheduling delle risorse).
  - Vantaggio: non occorre modificare il SO guest.
  - Svantaggio: degrado delle prestazioni rispetto a hypervisor di sistema
  - Esempi: Bochs, Parallels Desktop, VirtualBox.

# Full-Virtualization vs Para-Virtualization

Quale modalità di dialogo tra la VM e hypervisor per l'accesso alle risorse fisiche, ovvero come gestire l'esecuzione di istruzioni privilegiate?

- **Virtualizzazione completa**
- **Paravirtualizzazione**

## Virtualizzazione completa (full)

- L'hypervisor espone ad ogni VM interfacce hw simulate *funzionalmente identiche* a quelle della sottostante macchina fisica.
- L'hypervisor *intercetta* le richieste di accesso privilegiato all'hw (ad es. istruzioni di I/O) e ne emula il comportamento atteso.
- Esempi: KVM, VMware ESXi, Microsoft Hyper-V.

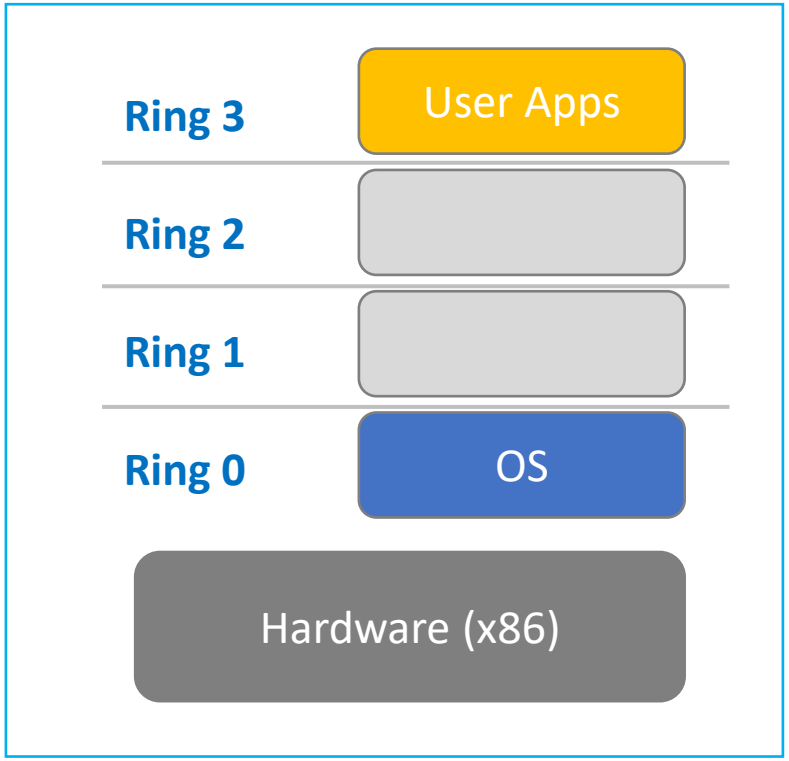
## Para-Virtualizzazione

- L'hypervisor espone ad ogni VM interfacce hw simulate *funzionalmente simili* (ma non identiche) a quelle della sottostante macchina fisica.
- Non viene emulato l'hw, ma viene creato uno strato minimale di sw (Virtual Hardware API) per assicurare la gestione delle VM ed il loro isolamento.
- Esempi: Xen, Oracle VM (basato su Xen), PikeOS.

# Requisiti per la Virtualizzazione dell'HW

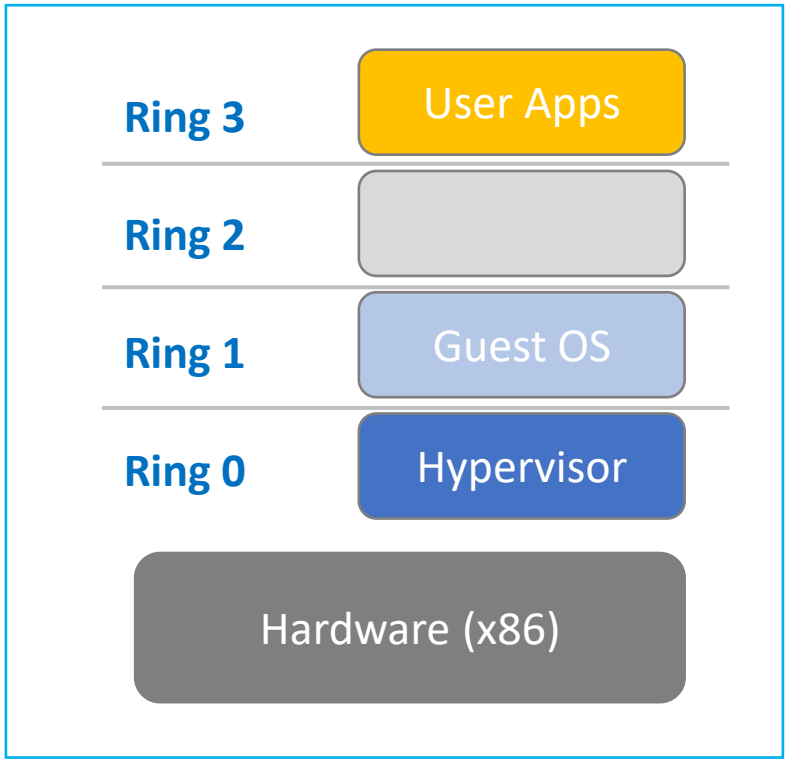
- Un SO è progettato per lavorare direttamente su hardware (bare-metal) e per controllare pienamente le risorse hardware.
  - In un sistema non virtualizzato, l'architettura x86 offre quattro livelli di privilegio:
    - Ring 0, 1, 2 e 3;
    - Le App utente lavorano in Ring 3 (privilegi minimi o modalità utente);
    - OS lavora in Ring 0 (privilegi massimi o modalità supervisor).
- La sfida della virtualizzazione dell'HW:
  - Istanziare il livello di virtualizzazione (hypervisor) sotto il SO;
  - Risulta difficile intercettare e tradurre le istruzioni privilegiate di SO in runtime.

[x86 è la classe di processor Intel (p.es. Pentium), qui presa come riferimento]



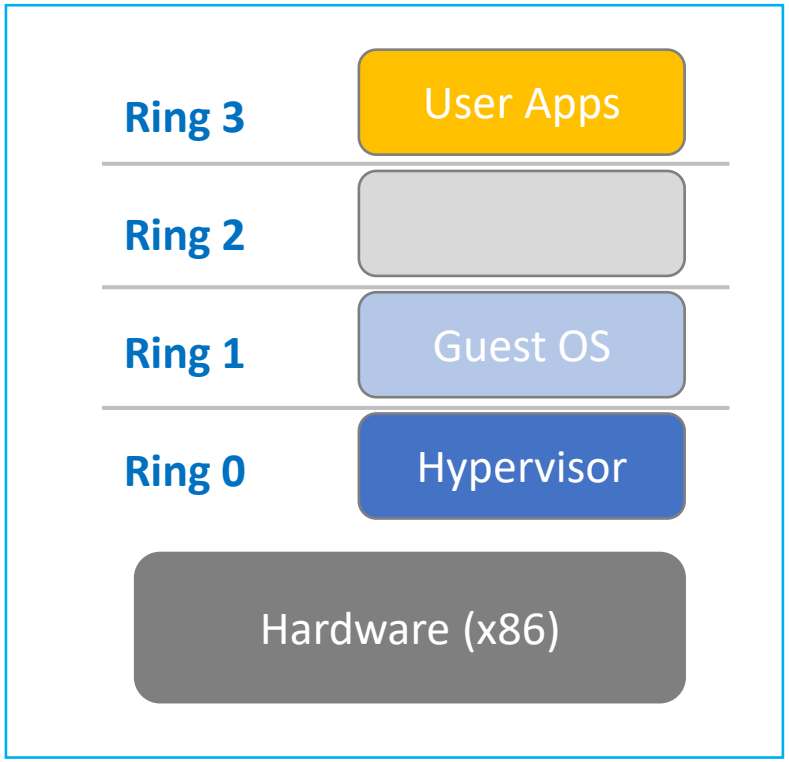
# Virtualizzazione Completa 1/3

- Il VMM opera in modalità supervisor (Ring 0).
- Il SO guest e le applicazioni (le VM) operano in user mode (Ring 3 o Ring 1 per SO guest).
- Il VMM disaccoppia il Guest OS dall'HW sottostante.
- Il Guest OS non sa di essere virtualizzato.
- Problema del **Ring Deprivileging**: il SO guest opera in un ring che non gli è proprio → non può eseguire istruzioni privilegiate (e.g., lidt in x86, load interrupt descriptor table).
- Problema del **Ring Compression**: poiché applicazioni e SO guest eseguono allo stesso livello, occorre proteggere lo spazio del SO.



# Virtualizzazione Completa 2/3

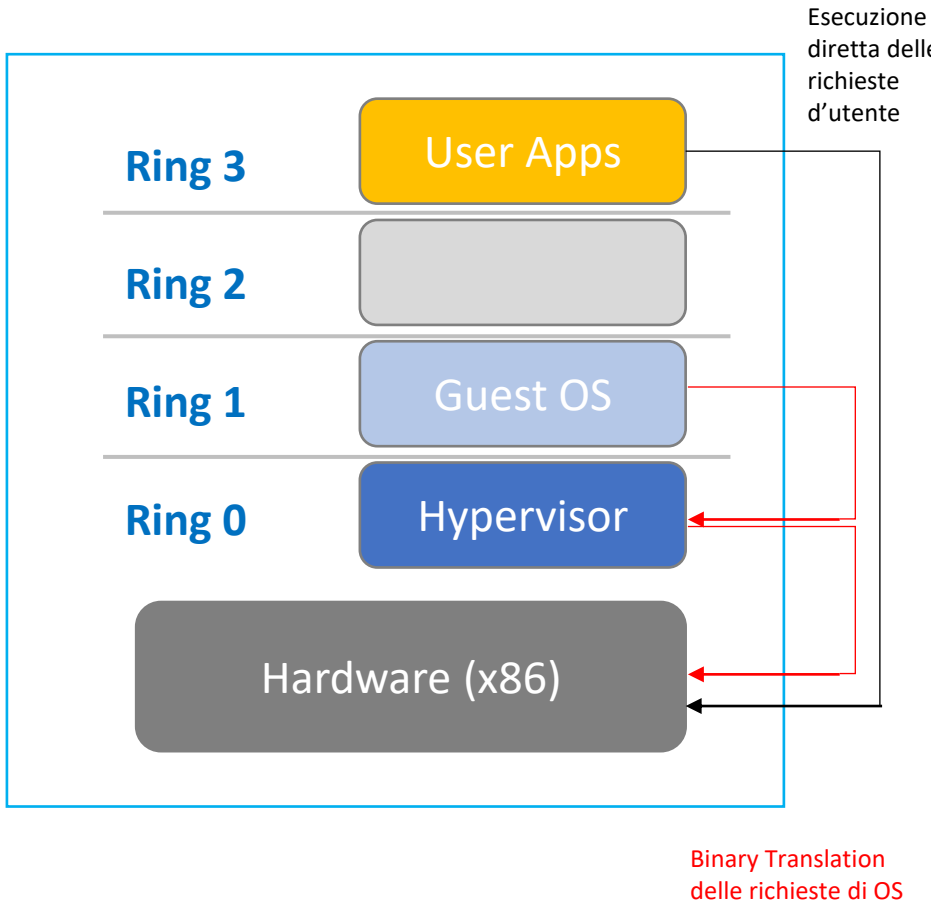
- Come si resolve il Ring Deprivileging?
  - Trap-and-emulate: quando il SO guest tenta di eseguire un'istruzione privilegiata, occorre notificare un'eccezione (trap) al VMM e trasferirgli il controllo; il VMM controlla la correttezza dell'operazione richiesta e ne emula il comportamento.
  - Le istruzioni non privilegiate richieste dal SO guest sono invece eseguite direttamente.
- Quando il meccanismo di trap è realizzato a livello software, si parla di (fast) binary translation.
- Nella Full Virtualization, ad ogni VM viene assegnato un VMM:
  - fornisce ad ogni VM i componenti virtualizzati;
  - esegue la Binary Translation (BT) delle istruzioni non-virtualizzabili.



# Virtualizzazione Completa 3/3

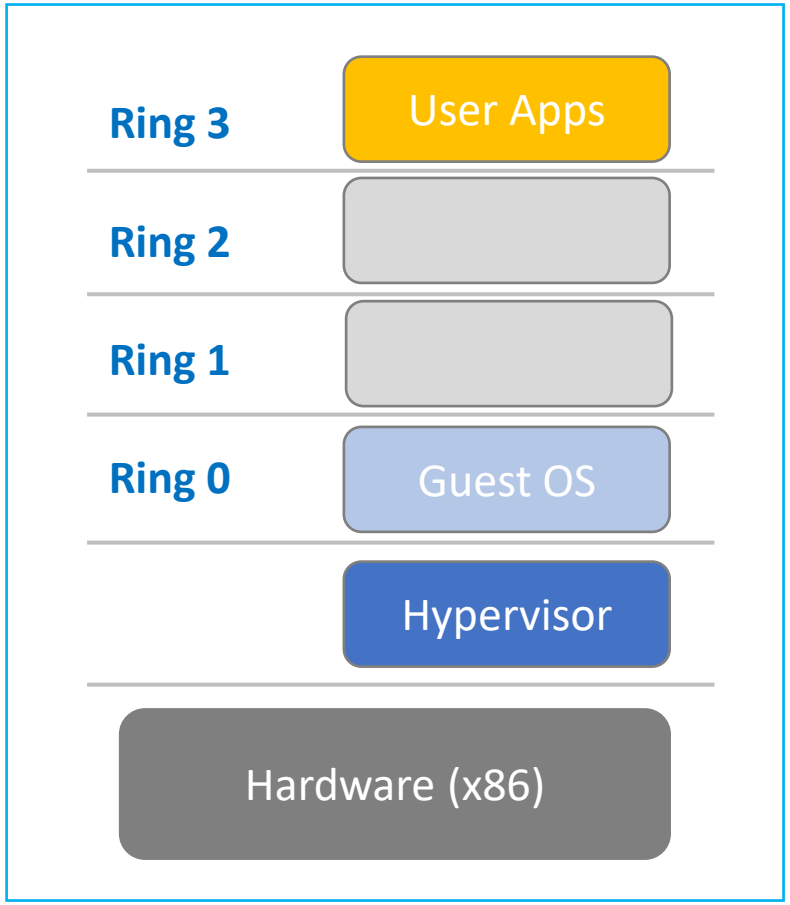
## (Fast) binary translation

- Il VMM scansiona il codice prima della sua esecuzione, al fine di sostituire blocchi contenenti istruzioni privilegiate con blocchi funzionalmente equivalenti ma contenenti istruzioni per la notifica di eccezioni al VMM.
- I blocchi tradotti sono eseguiti direttamente sull'hw e conservati in una cache per eventuali riusi futuri.
- Maggiore complessità del VMM e minori prestazioni.



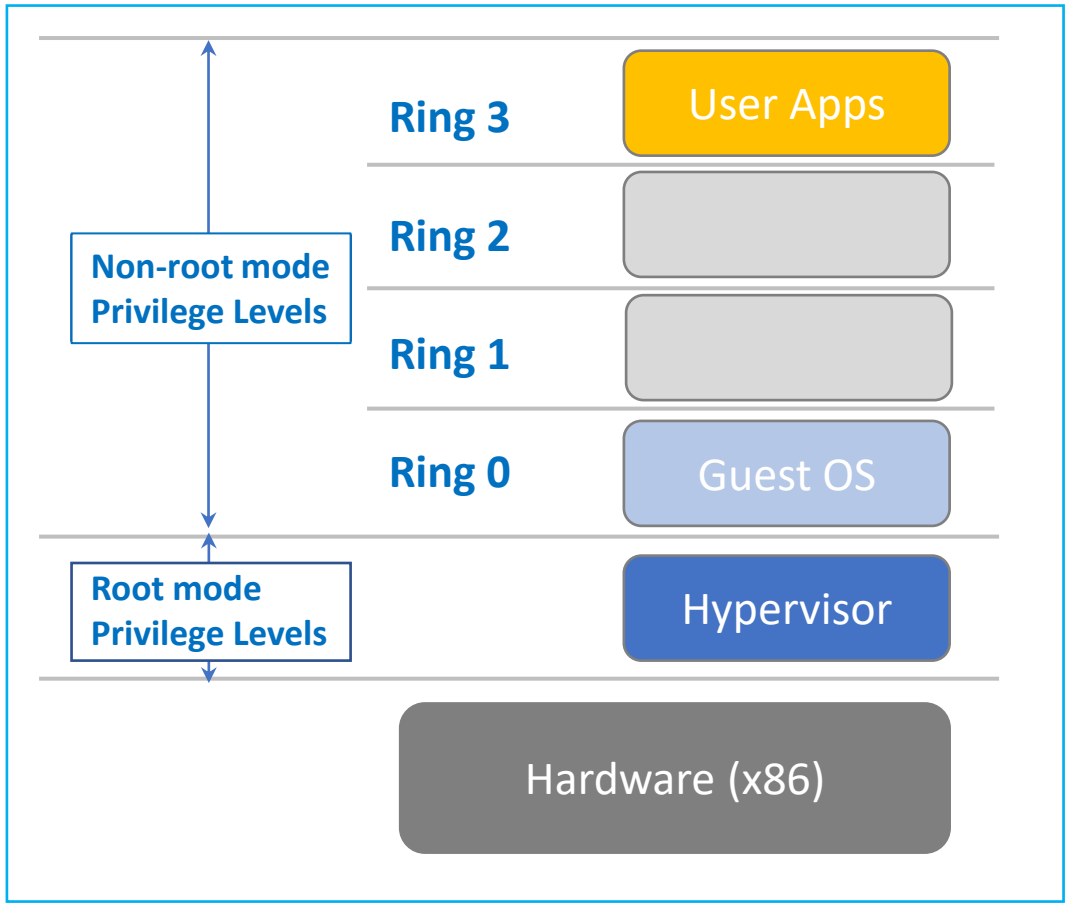
# Hardware-assisted CPU Virtualization 1/2

- Quando il meccanismo di trap è realizzato a livello hardware, si parla di hardware-assisted CPU virtualization (o accelerated virtualization).
- In questo caso, il processore fornisce supporto alla virtualizzazione: si utilizza una CPU hypervisor-aware, allo scopo di gestire le istruzioni privilegiate.
- I processori che supportono questa funzionalità sono Intel VT-x e AMD-V, mentre, p.es. IA-32 non la supporta.



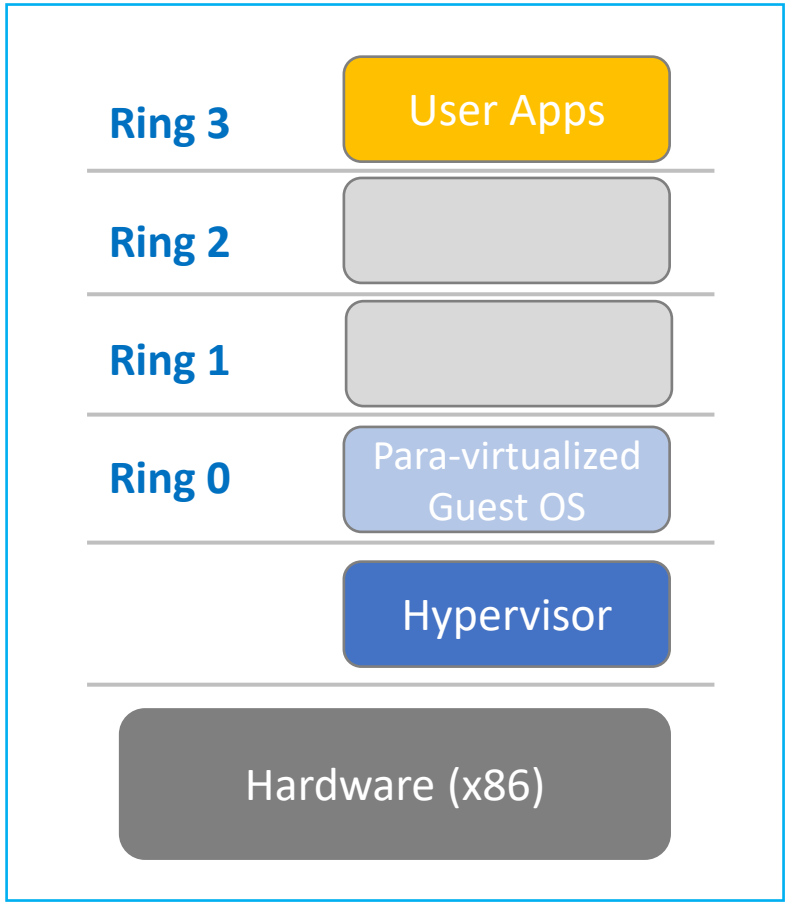
# Hardware-assisted CPU Virtualization 2/2

- Si definiscono in questo caso due nuove modalità operative per la CPU, chiamate root mode e non-root mode, entrambe che supportano tutti i quattro Ring di privilegio su x86.
- VMM opera in root-mode (root-Ring 0), mentre tutti gli OS Guest operano in guest mode nei loro livelli di privilegio originari (non-root mode Ring 0).
- Così facendo, non ci sono più i problemi di ring compression e ring deprivileging.
- VMM può controllare l'esecuzione dei processi a livello guest attraverso il controllo di strutture dati in memoria.



# Para-Virtualizzazione 1/4

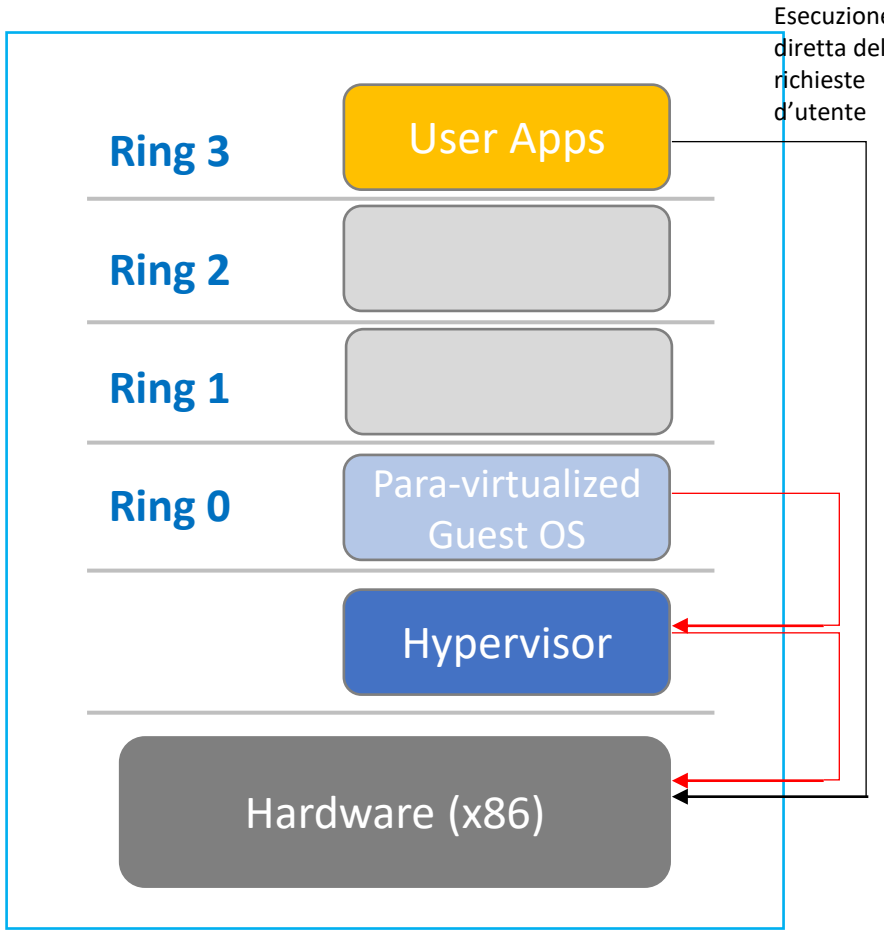
- Il Guest OS sa di essere virtualizzato.
- Il Guest OS opera sul Ring 0.
- Serve utilizzare un Guest OS kernel modificato, per eliminare la necessità della binary translation.
- Le modifiche del Guest OS sono possibili solo nel caso di sistemi open source, come Linux e OpenBSD.
- Risulta impossible modificare un OS "chiuso" come Microsoft Windows, per cui non si può realizzare la para-virtualizzazione.



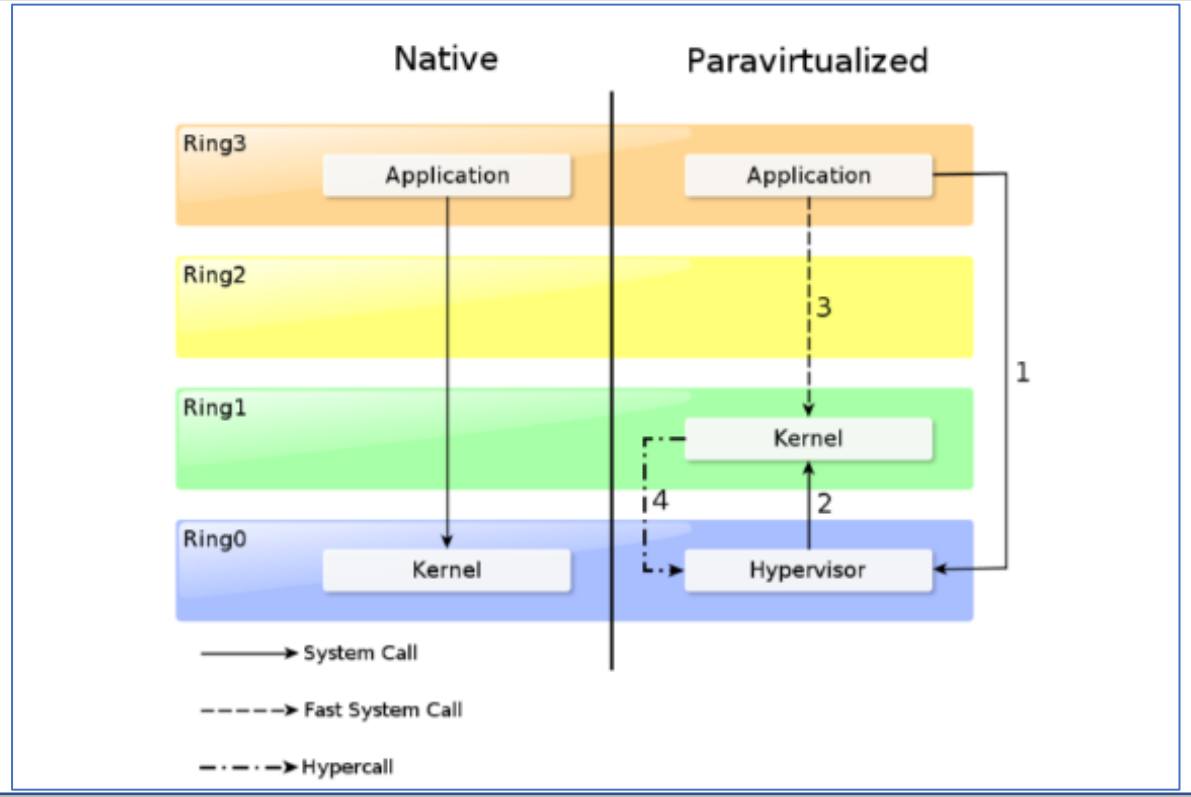
# Para-Virtualizzazione 2/4

Hypercall verso il livello di virtualizzazione che sostituiscono le istruzioni OS non-virtualizzabili

- La Para-virtualizzazione è quindi un metodo non trasparente:
  - Il kernel del Guest OS deve essere modificato per permettergli di invocare le API virtuali esposte da hypervisor.
- Le istruzioni non-virtualizzabili sono sostituite da hypercall, che comunicano direttamente con l'hypervisor.
  - Una sorta di system call dal kernel del Guest OS verso hypervisor
  - *hypercall : hypervisor = system call : kernel*



# Para-Virtualizzazione 3/4



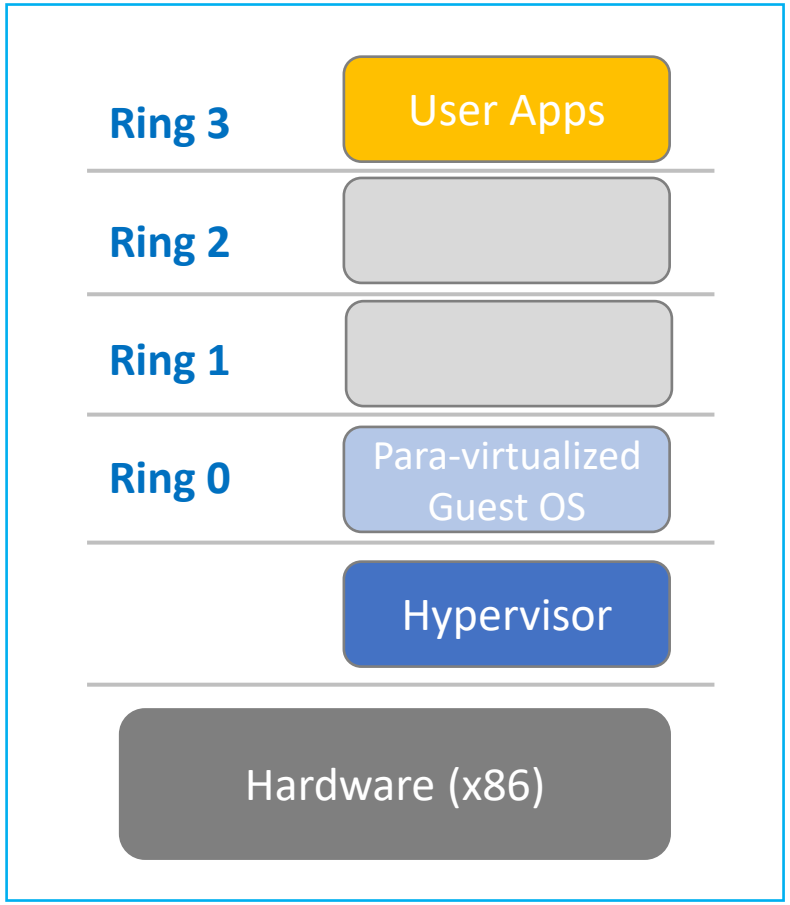

Hypercall execution

- When an application running in the VM issues a guest OS system call, through the hypercall the control flow jumps to hypervisor, which then passes control back to the guest OS.

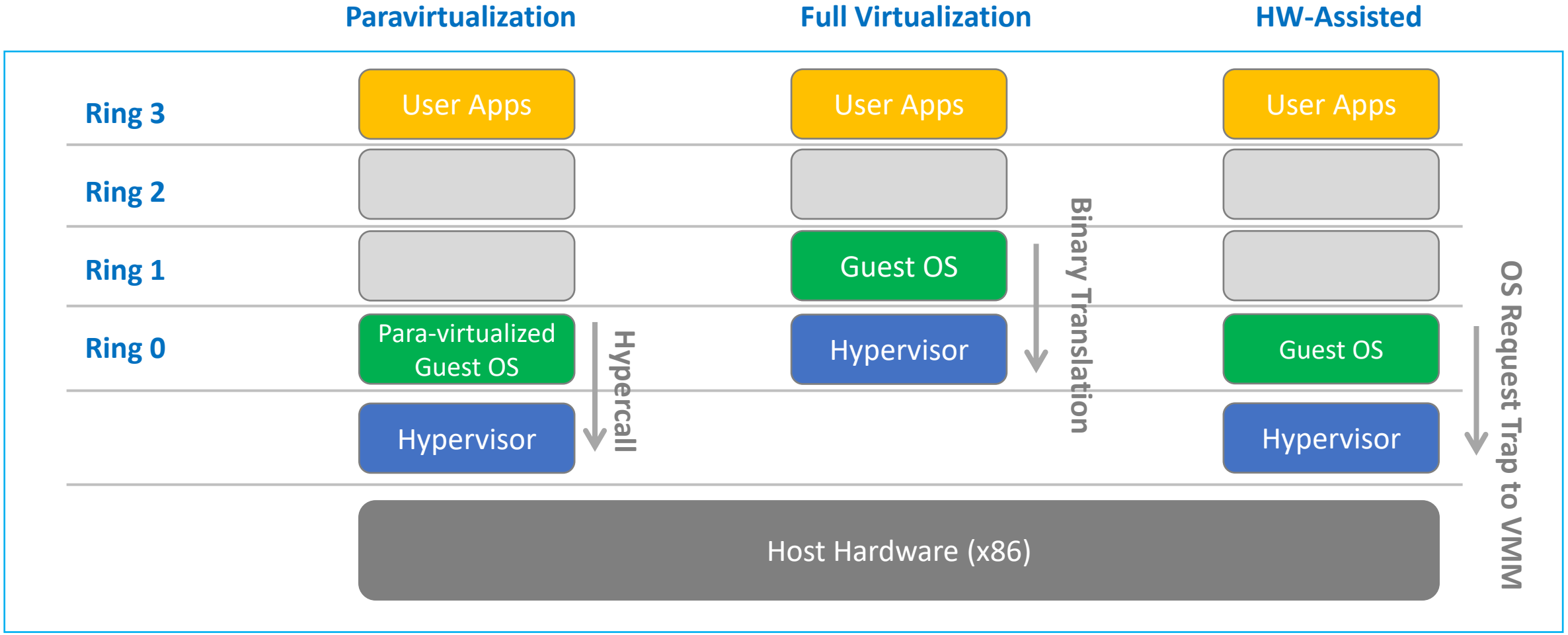
Source: **The definitive Guide to XEN hypervisor**, Prentice Hall.

# Para-Virtualizzazione – Vantaggi e Svantaggi 4/4

- Pro (vs. Full Virtualization)
  - Relativamente più semplice e più pratico nell'implementazione.
  - Più efficiente perchè con ridotto overhead rispetto a fast binary translation.
  - Non richiedere estensioni specifiche della CPU (come invece l'HW-assisted virtualization).
- Contra
  - Richiedere la disponibilita del codice sorgente del OS, al fine di poterlo modificare.
    - ✓ I SO chiusi (p.es. Windows) potrebbero usare ad-hoc driver che rimappano l'esecuzione di istruzioni privilegiate sulla virtual API esposta da VMM (soluzione complessa).
  - Costi di mantenimento dei SO para-virtualizzati
    - I SO para-virtualizzati non possono operare direttamente su HW.



# Riassumendo i diversi approcci di Virtualizzazione




**Xen, KVM**

**VMware ESX/ESXi, Microsoft Hyper-V**

**Intel VT-x, AMD-V**